Topic 16 - Production Workflow Planning

Step 1: Mock Billing API Setup (Choose One of 3 Options)

To test the external billing charge step without requiring a paid merchant gateway (Stripe/Authorize.net), select one of the following mock options:

Option A: Free Cloud Mock API via DummyJSON Custom Response (Recommended - Zero Install)

DummyJSON Custom Response generates an instant, persistent cloud mock REST API returning customized JSON:

1. Navigate to **https://dummyjson.com/custom-response** in your browser. **2.** In the **JSON Response** editor, paste the mock charge response payload:

```
{
 "chargeId": "ch_dummyjson_882910",
 "status": "succeeded",
 "amount": 249.99,
 "currency": "USD",
 "timestamp": "2026-10-15T12:00:00.000Z"
}
```

3. Set **HTTP Status Code** to `201 Created` (or `503 Service Unavailable` when testing failure retry backoff).

4. Click **Generate API URL**.

5. Copy the generated live endpoint URL (e.g. `https://dummyjson.com/c/your-unique-id`). **6.** In n8n, paste this URL into `HTTP_PostBillingCharge` as the target URL.

Option B: Postman Mock Server (Cloud Mock Endpoint)

If you use Postman for API testing and team collaboration:
1. Open **Postman** and create a new Collection named `n8n Intermediate Assessment`.

2. Add a new Request:

Method: `POST`

Path / Name: `/v2/charges`

3. In the top right of the request pane, click **...** $\rightarrow$ **Add Example**.
4. In the Example configuration:

   Set Status Code to `201 Created`.

   Set Body (JSON):

```
{
 "chargeId": "ch_postman_mock_4421",
 "status": "succeeded",
 "amount": 249.99,
 "currency": "USD"
}
```

5. Save the Example.
6. In the left sidebar, click `...` on the `n8n Intermediate Assessment` collection $\rightarrow$ select **Mock Collection**.
7. Name the mock server `n8n Billing Mock` and click **Create Mock Server**.
8. Copy the generated **Postman Mock URL** (e.g. `https://{{mock-id}}.mock.pstmn.io/v2/charges`).
9. In n8n, paste this Postman mock URL into `HTTP_PostBillingCharge`.

Option C: In-n8n Local Webhook Mock Server (Offline Sandbox)

If running in an isolated or offline local environment without internet access:

1. In n8n, create a new workflow named `Mock Billing Server`.
2. Add a **Webhook Node**:

   Node Name: `Mock_Billing_Webhook`

   HTTP Method: `POST`

   Path: `mock/v2/charges`

   Response Mode: `Using 'Respond to Webhook' Node`

3. Connect an **IF Node** named `IF_SimulateOutage`:

   Value 1: `={{ $json.body.amount }}`

   Operation: `Equal`

   Value 2: `999`

   True Branch: Connect to **Respond to Webhook Node**:

   Response Code: `503`

   Response Body: `{"error": "Billing Gateway Temporarily Unavailable", "code": "GATEWAY_TIMEOUT"}`

False Branch: Connect to **Respond to Webhook Node**:

Response Code: `201`

Response Body (Expression):

```
{{

              {
               "chargeId": "ch_mock_" +
             Math.random().toString(36).substring(2, 10),  "status":
             "succeeded",
               "amount": $json.body.amount,
               "currency": $json.body.currency,
               "timestamp": new Date().toISOString()
               }
}}
```

4. Save and activate the workflow. (Recommendation: Use Option A via DummyJSON Custom Response or Option B via Postman for instant, zero-blocker testing without port configuration).

Step 2: Configure Main Ingestion Webhook (`Webhook_Ingest`)

1. **Create Workflow:**

   Open your n8n dashboard (e.g. `http://localhost:5678` or hosted cloud instance). Click **+ Add workflow** in the top-right corner of the canvas.

   Click on the workflow name in the top-left corner (default `My workflow 1`) and rename it to: `Topic 1 - Production Workflow Planning`.

2. **Add Node to Canvas:**

   Click the **+ Add first step...** button on the empty canvas (or press **Tab**).

   In the node search panel on the right, type `Webhook`.

   Under the **Trigger** section, click **Webhook** to place it on the canvas.

3. **Rename the Node:**

   Double-click the node to open its configuration panel.

   Click the pencil icon next to the node title at the top-left (or double-click the name) and enter: `Webhook_Ingest`.

4. **Configure Parameters (Parameters Tab):**

   HTTP Method: Click the dropdown and select `POST`.

   Path: Enter `v1/transactions` into the text field (Mode: **Fixed**).

   Authentication: Select `None`.

   Response Mode: Click the dropdown and select `Immediately`.

   Response Code: Enter `202` (Mode: **Fixed**).

   Response Data: Click the dropdown and select `JSON`.

Response Body: In the JSON text editor, enter:

```
{
 "message": "Transaction accepted for processing",
 "status": "ACCEPTED"
}
```

5. **Configure Environment Endpoints:**

   Notice n8n provides two webhook URLs in the panel:

   Test URL: e.g., `https://n8n.tayanasolutions.com/webhook-test/v1/transactions` (Used for real-time canvas testing when clicking 'Listen for test event').

   Production URL: e.g., `https://n8n.tayanasolutions.com/webhook/v1/transactions` (Used when the workflow is toggled Active).

6. **Test Node Checkpoint:**

   Click the **Listen for test event** button at the bottom of the modal.

   Send a test POST request via cURL or Postman to the **Test URL** with body:

```
{
 "transactionId": "TX-10001",
 "customerEmail": "sarah.connor@cyberdyne.io",
 "amount": 249.99,
 "currency": "USD",
 "description": "Enterprise Cloud Subscription"
}
```

   Verify the webhook receives the payload and displays the parsed JSON body in the **Output** pane on the right.

Step 3: Implement Idempotency Verification (`Code_CheckIdempotency`)

1. **Add Node to Canvas:**

   Hover over the right-side connector dot (output handle) of `Webhook_Ingest` and click the `+` icon (or drag out a connection line and release).

   In the search panel, type `Code` and select the **Code** node.

2. **Rename the Node:**

   Click the pencil icon on the top-left of the modal and rename the node to: `Code_CheckIdempotency`.

3. **Configure Parameters (Parameters Tab):**

   Language: Select `JavaScript` from the dropdown.

   Mode: Select `Run Once for Each Item`.

   JavaScript Code: Paste the following caching script into the code editor:

```
// Global memory cache simulation inside n8n workflow static data
const staticData = $getWorkflowStaticData('global');
staticData.processedTransactions = staticData.processedTransactions || [];

const body = $input.item.json.body || $input.item.json;
const transactionId = String(body.transactionId || '').trim();

if (!transactionId) {

 return {
 json: {
 isDuplicate: false,
 hasId: false,
 payload: body,
 errorMsg: 'Missing required transactionId'
 }
 };
}

// Check if transaction ID already exists in cache
const isDuplicate = staticData.processedTransactions.includes(transactionId);

if (!isDuplicate) {
 // Store key with limit to prevent memory bloat
 staticData.processedTransactions.push(transactionId);
 if (staticData.processedTransactions.length > 500) {
 staticData.processedTransactions.shift(); // Evict oldest
 }
}

return {
 json: {
 transactionId: transactionId,
 isDuplicate: isDuplicate,
 hasId: true,
 payload: body
 }
};
```

4. **Test Node Checkpoint:**

   Click the **Test step** button at the bottom of the node panel.

   In the **Output** panel, verify that `isDuplicate: false`, `hasId: true`, and `transactionId: "TX-10001"` appear in the JSON output.

Step 4: Add Duplicate Routing (`IF_IsNewTransaction`)

1. **Add Node to Canvas:**

   Hover over the right output handle of `Code_CheckIdempotency`, click the `+` icon, search for `If` , and select the **If** node.

2. **Rename the Node:**

   Double-click the header title and rename to: `IF_IsNewTransaction`.

3. **Configure Parameters (Parameters Tab):**

   Click **Add Condition** (or select the first condition row).

   Value 1: Hover over the text field, click the **Expression (`fx`)** toggle button in the upper-right corner of the box, and enter:

   ```
   ={{ $json.isDuplicate }}
   ```

   Operation: Click the comparison dropdown, select `Boolean` $ightarrow$ `Equal`. **Value 2:** Hover over the input, toggle to **Expression (`fx`)**, and enter:

   ```
   {{ false }}
   ```

   (Alternatively, leave in Fixed mode and choose `false` from the boolean dropdown).

4. **Connect False Branch (Duplicate Bypass Logger):**

   In n8n, the IF node has two outputs on the right side:

   Top output handle: `true` (New Transaction)

   Bottom output handle: `false` (Duplicate Transaction)

   Click the **false (bottom) output handle** of `IF_IsNewTransaction`, click `+`, search for `Code`, and rename it to: `Log_BypassDuplicate`.

   Under **Mode**, select `Run Once for Each Item`.

   In the **JavaScript Code** editor, paste:

   ```
   return {
    json: {
    event: "TRANSACTION_BYPASSED",
    reason: "Duplicate transactionId detected",
    transactionId: $json.transactionId,
    timestamp: new Date().toISOString()
    }
   };
   ```

   Click **Test step** to verify the bypass log format.

Step 5: Implement Schema Validation (`Code_ValidateSchema`)

1. **Add Node to Canvas:**

   Click the **true (top) output handle** of `IF_IsNewTransaction`, click the `+` icon, search for `Code`, and place a new Code node.

2. **Rename the Node:**

   Rename the node to: `Code_ValidateSchema`.

3. Configure Parameters (Parameters Tab):

Language: Select `JavaScript`.

Mode: Select `Run Once for Each Item`.

JavaScript Code: Paste the following validation script:

```
const payload = $json.payload || {};
const errors = [];

// 1. Email validation using standard RFC 5322 regex
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!payload.customerEmail || !emailRegex.test(payload.customerEmail)) {
 errors.push("Invalid or missing 'customerEmail'");


}

// 2. Amount validation (must be positive numerical value)
const amount = Number(payload.amount);
if (isNaN(amount) || amount <= 0) {
 errors.push("Field 'amount' must be a positive number greater than 0");
}

// 3. Currency validation (ISO 4217: 3 uppercase letters)
const currencyRegex = /^[A-Z]{3}$/;
if (!payload.currency || !currencyRegex.test(payload.currency)) {
 errors.push("Field 'currency' must be a valid 3-letter uppercase ISO code (e.g. USD, EUR, GBP)"); }

const isValid = errors.length === 0;

return {
 json: {
 transactionId: $json.transactionId,
 isValid: isValid,
 validationErrors: errors,
 sanitizedPayload: {
 transactionId: $json.transactionId,
 customerEmail: String(payload.customerEmail || '').toLowerCase().trim(),
 amount: amount,
 currency: String(payload.currency || '').toUpperCase().trim(),
 description: payload.description || 'Standard Order Charge'
 }
 }
};
```

4. Test Node Checkpoint:

Click **Test step** at the bottom of the panel.

In the Output pane, verify that `isValid: true`, `validationErrors: []`, and the normalized `sanitizedPayload` object are successfully generated.

Step 6: Route Validation Status (`IF_IsValidPayload`)

1. Add Node to Canvas:

Hover over the right output handle of `Code_ValidateSchema`, click `+`, search for `If`, and select the **If** node.

2. Rename the Node:

Rename to: `IF_IsValidPayload`.

3. Configure Parameters (Parameters Tab):

Click **Add Condition**.

Value 1: Switch toggle to **Expression (`fx`)** and enter:

```
={{ $json.isValid }}
```

Operation: Select `Boolean` $ ightarrow$ `Equal`.

Value 2: Switch to **Expression (`fx`)** and enter `{{ true }}` (or select Fixed `true`). All rights

4. Connect False Branch (Validation Error Logger):

Click the **false (bottom) output handle** of `IF_IsValidPayload`, click `+`, search for `Code`, and rename to: `Log_ValidationError`.

Under **Mode**, select `Run Once for Each Item`.

Paste the error logging script:

```
return {
 json: {
 event: "VALIDATION_FAILED",
 transactionId: $json.transactionId,
 errors: $json.validationErrors,
 timestamp: new Date().toISOString()
 }
};
```

Click **Test step** to verify.

Step 7: Call External Billing API with Retries (`HTTP_PostBillingCharge`)

1. **Add Node to Canvas:**
   Hover over the **true (top) output handle** of `IF_IsValidPayload`, click `+`, search for `HTTP Request`, and place the node.
2. **Rename the Node:**
   Rename the node to: `HTTP_PostBillingCharge`.
3. **Configure Parameters (Parameters Tab):**
   Method: Click the dropdown and select `POST`.

URL: Click the input field, toggle from Fixed to **Expression (`fx`)**, and paste your endpoint URL:

DummyJSON Option A: `https://dummyjson.com/c/<your-unique-id>`

Postman Option B: `https://{{mock-id}}.mock.pstmn.io/v2/charges`

Local Webhook Option C: `http://localhost:5678/webhook/mock/v2/charges`

Authentication: Select `None`.

Send Headers: Toggleto `OFF` (or add `Content-Type: application/json` if testing against custom backends).

Send Body: Switch the toggle to `ON`.

Body Content Type: Click the dropdown and select `JSON`.

Specify Body: Click the dropdown and select `Using JSON`.

JSON: Click inside the editor box, switch to **Expression (`fx`)** mode, and enter:

```
{{
 {
 "transactionId": $json.sanitizedPayload.transactionId,
 "customerEmail": $json.sanitizedPayload.customerEmail,
 "amount": $json.sanitizedPayload.amount,


 "currency": $json.sanitizedPayload.currency,
 "description": $json.sanitizedPayload.description
 }
}}
```

4. **Configure Fault-Tolerance & Retry Policies (Settings Tab):**

   Look at the top navigation bar of the node modal and click the **Settings** tab.

   Retry on Fail: Switch the toggle to `ON`.

   Max Tries: Enter `3` (retries up to 3 times before failing).

   Wait Between Tries (ms): Enter `5000` (5-second exponential cooldown).

   Never Error: Keep `OFF` (ensures fatal gateway timeouts bubble up to the centralized Error Trigger).

5. **Test Node Checkpoint:**

   Switch back to the **Parameters** tab.

   Click **Test step** at the bottom of the modal.

   Verify the response in the Output pane displays HTTP status `200` or `201` with `chargeId` (e.g. `ch_dummyjson_882910`).

Step 8: Build Success Audit Record (`Code_FormatSuccessRecord`)

1. **Add Node to Canvas:**

Hover over the right output handle of `HTTP_PostBillingCharge`, click `+`, search for `Code`, and add the node.

2. **Rename the Node:**

   Rename to: `Code_FormatSuccessRecord`.

3. **Configure Parameters (Parameters Tab):**

   Language: Select `JavaScript`.

   Mode: Select `Run Once for Each Item`.

   JavaScript Code: Paste the following audit compilation code:

```
const response = $input.item.json;
return {
 json: {
 status: "PROCESSED",
 chargeId: response.chargeId || 'ch_generated',
 transactionId: response.transactionId ||
$node['Code_ValidateSchema'].json.sanitizedPayload.transactionId,  amountCharged: response.amount,
 currency: response.currency,
 completedAt: new Date().toISOString()
 }
};
```

4. **Test Node Checkpoint:**

   Click **Test step**.

   In the Output pane, verify that `status: "PROCESSED"` and the complete audit object are generated.

Step 9: Centralized Error Handler Configuration (`Error_Trigger` & `Code_BuildIncidentAlert`)

1. **Add Error Trigger to Canvas:**

   Click on an empty canvas area separate from the primary transaction chain.

   Click `+` (or press **Tab**), search for `Error Trigger`, and place it.

   Double-click the node and rename it to: `Error_Trigger`.

   Note: The Error Triggernode automatically activates whenever any node in the workflow exhausts its retry attempts and fails.

2. **Connect Incident Diagnostics Formatter:**

   Hover over the right connector handle of `Error_Trigger`, click `+`, search for `Code`, and rename to: `Code_BuildIncidentAlert`.

   Language: Select `JavaScript`.

   Mode: Select `Run Once for All Items`.

   JavaScript Code: Paste the diagnostic extraction script:

```
const errorData = $input.first().json;
return {
 json: {
 alertType: "CRITICAL_PIPELINE_INCIDENT",
 workflowId: $workflow.id,
 workflowName: $workflow.name,
```

```
 failedNode: errorData.execution.error.node?.name || "Unknown Node",
 errorMessage: errorData.execution.error.message || "Unspecified execution
failure",  executionUrl: errorData.execution.url,
 timestamp: new Date().toISOString()
 }
};
```

3. **Test Node Checkpoint:**

   Click **Test step** on `Code_BuildIncidentAlert`.

   Verify that the incident alert structure correctly displays `workflowName`, `failedNode`, and `errorMessage`.